Name: Ruturaj Amit Rajwade
Class: D12C
Roll No.: 46

Practical: Implementing Automata and Formal Languages with JFLAP 7.1

Aim: To implement and test a range of automata, grammars and formal-language concepts using JFLAP 7.1.

Software Requirements

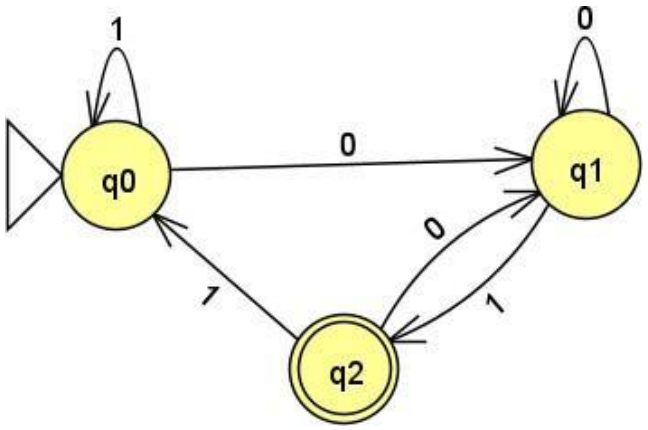
- JFLAP 7.1
- Java Runtime Environment (JRE) / JDK

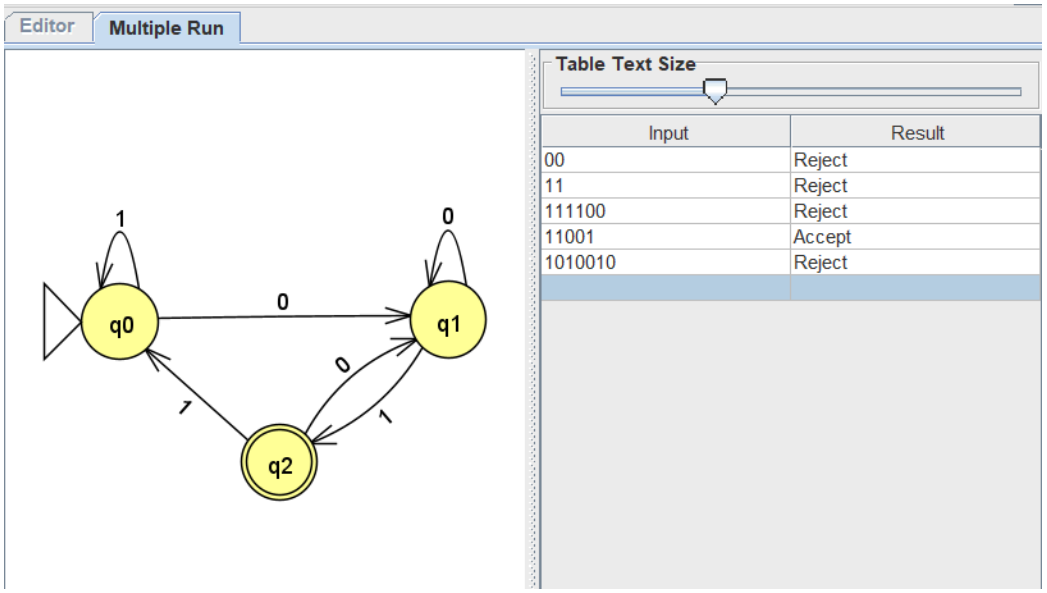
Question 1: Deterministic Finite Automaton (DFA)

Aim: To design a DFA that accepts all binary strings ending in 01.

Procedure

1. Open JFLAP and choose Finite Automaton.
2. Create the states and transitions needed to recognise strings that end in 01.
3. Mark the initial state and the final state(s).
4. Run sample strings through the automaton and note whether each is accepted or rejected.





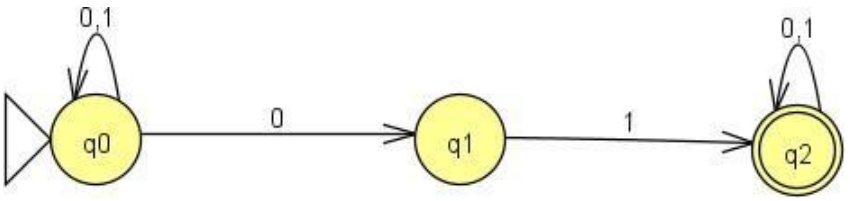
Result: The DFA was designed and correctly accepts binary strings ending in 01.

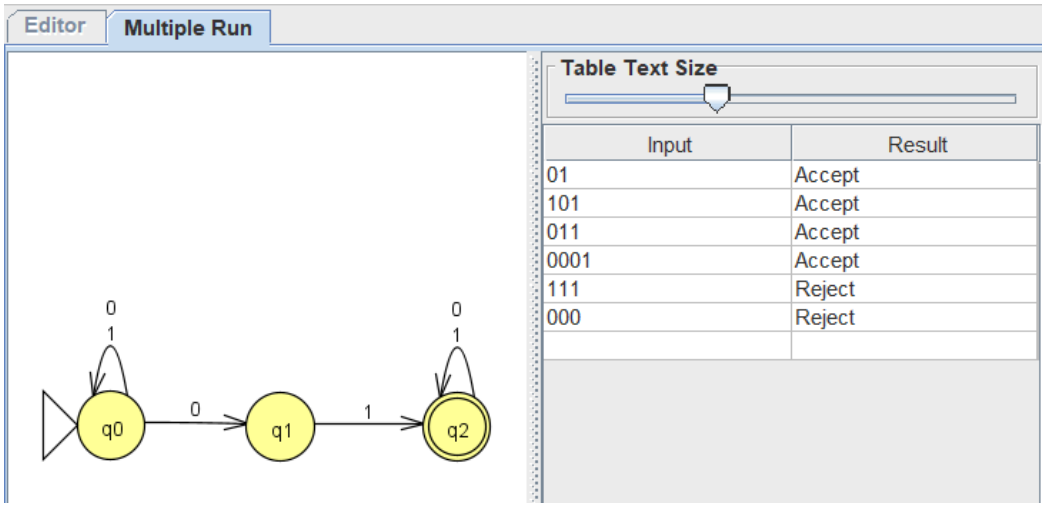
Question 2: Nondeterministic Finite Automaton (NFA)

Aim: To design an NFA that accepts all strings containing 01 as a substring.

Procedure

1. Create a new NFA in JFLAP.
2. Add the required states, transitions and self-loops.
3. Mark the initial and final states.
4. Test strings that contain 01 and strings that do not.





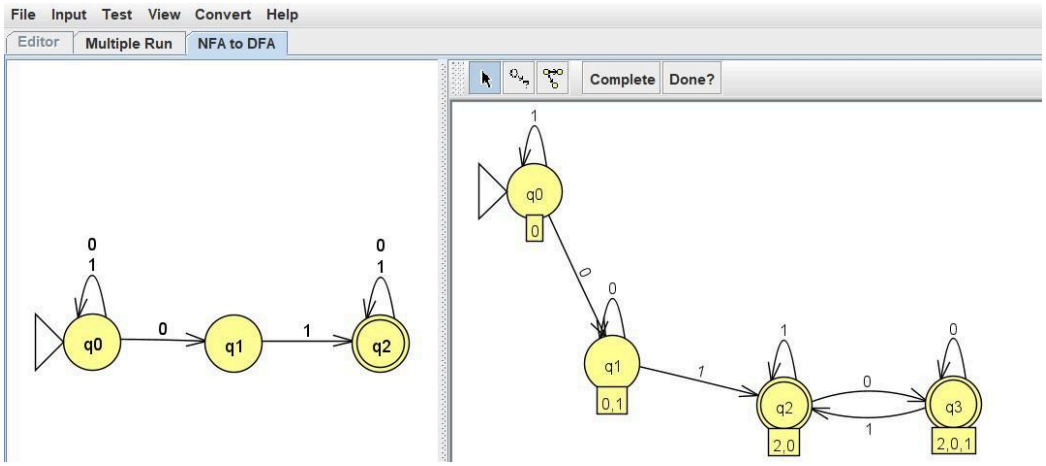
Result: The NFA was designed and accepts strings containing 01.

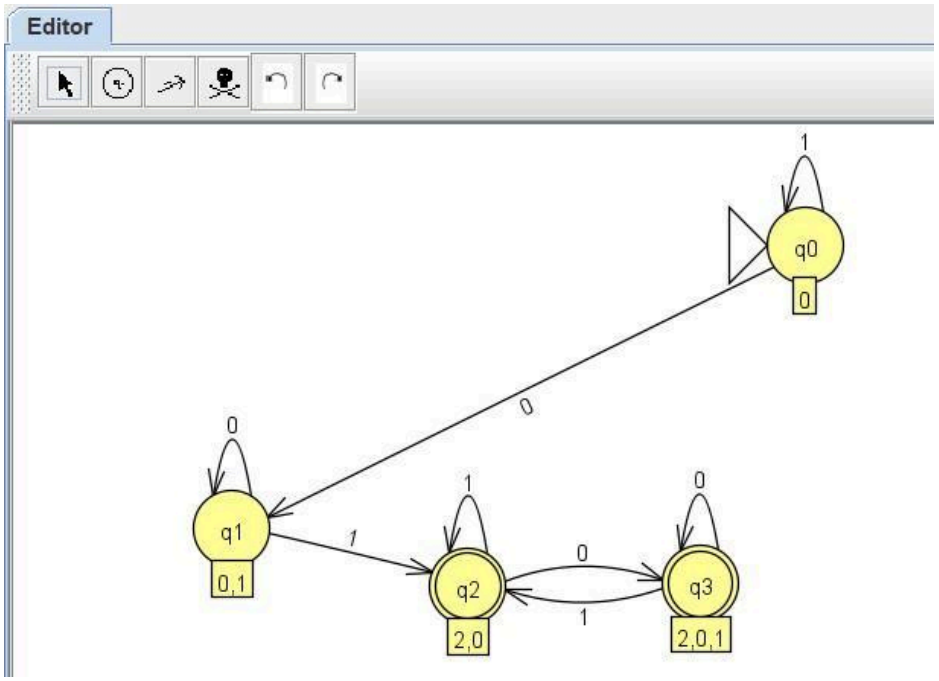
Question 3: Converting an NFA to an Equivalent DFA

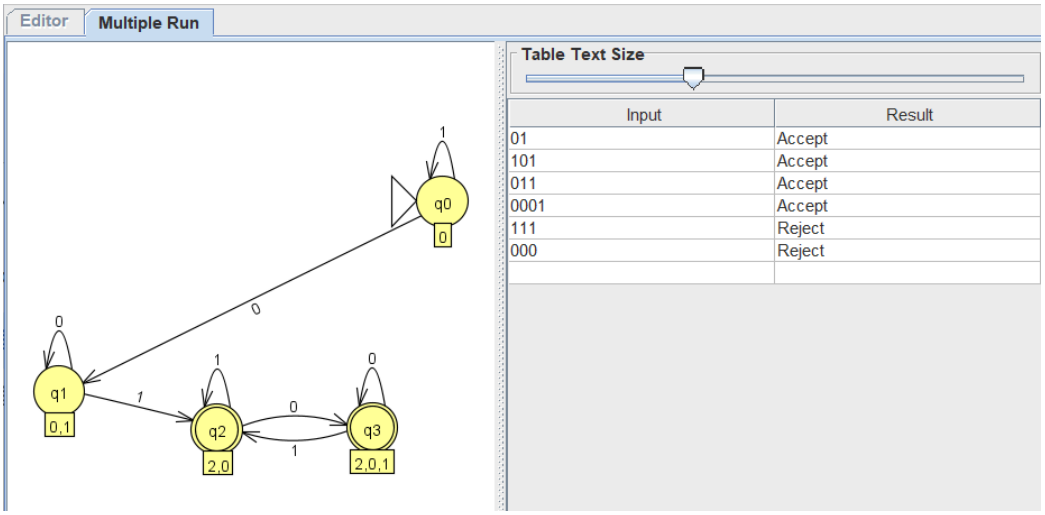
Aim: To convert an NFA into an equivalent DFA using JFLAP.

Procedure

1. Open the NFA created in Question 2.
2. Choose the NFA-to-DFA conversion option.
3. Examine the DFA that JFLAP generates.
4. Run the same strings on both automata and compare the outcomes.







Result: The NFA was converted into an equivalent DFA, and both machines gave the same results on the test strings.

Question 4: Converting a Regular Grammar to a Finite Automaton

Aim: To convert a right-linear regular grammar into an equivalent finite automaton.

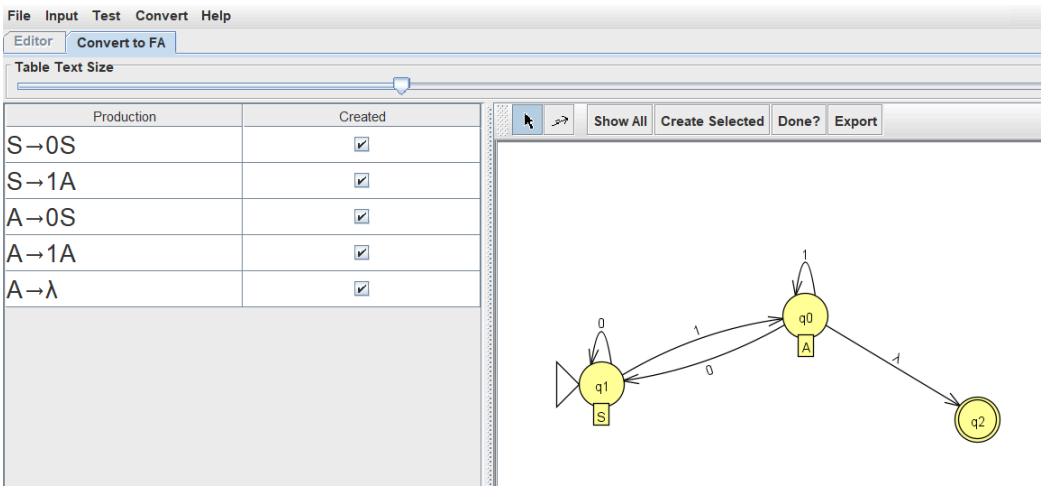
Grammar

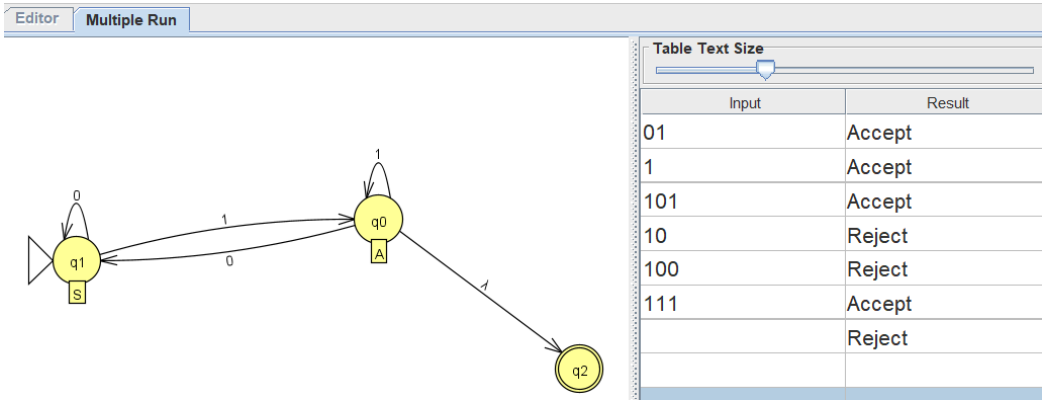
$S \rightarrow 0S \mid 1A$

$A \rightarrow 0S \mid 1A \mid \lambda$

Procedure

1. Open the Grammar editor in JFLAP.
2. Enter the production rules.
3. Convert the grammar into a finite automaton.
4. Test strings that end in 1 and strings that do not.





Result: The regular grammar was converted into an equivalent finite automaton.

Question 5: Context-Free Grammar (CFG)

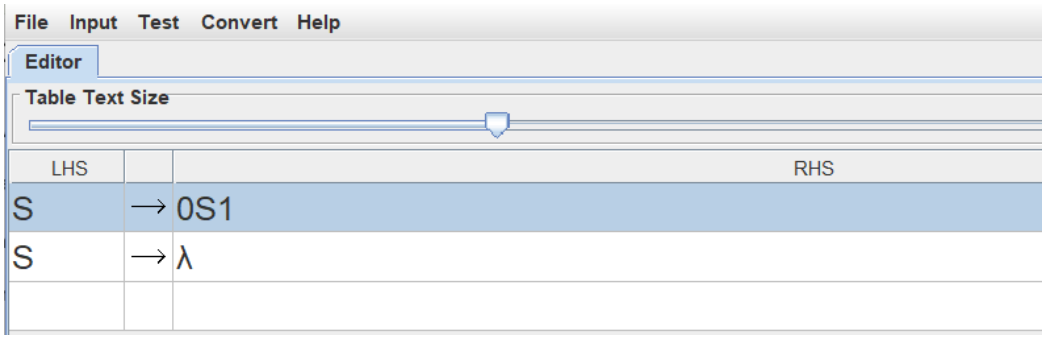
Aim: To construct a CFG for the language $L = \{0^n1^n \mid n \geq 0\}$.

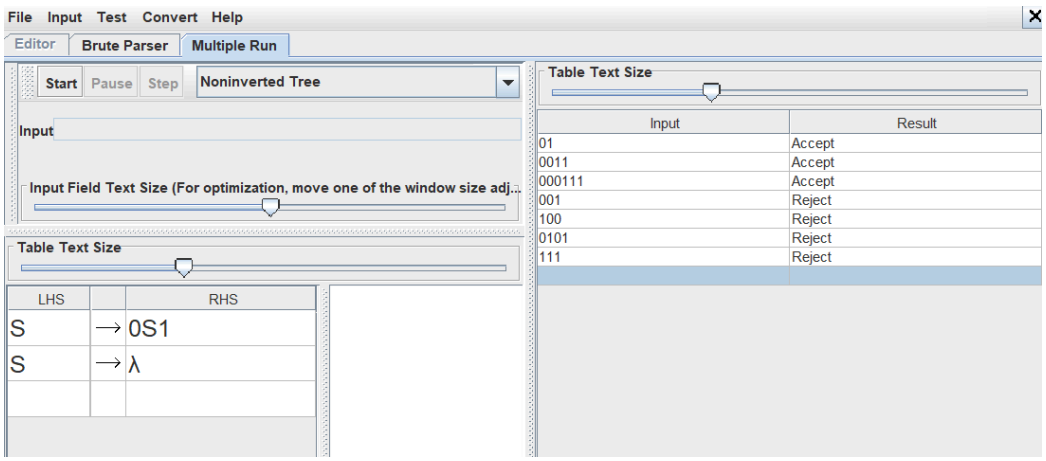
Grammar

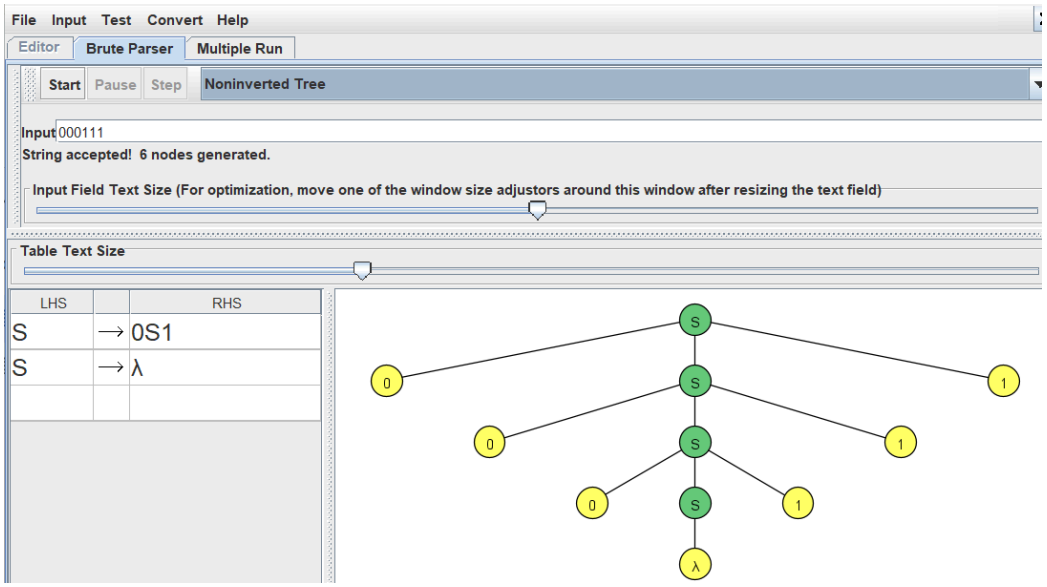
$$S \rightarrow 0S1 \mid \lambda$$

Procedure

1. Select Grammar in JFLAP.
2. Enter the production rules.
3. Generate strings from the grammar.
4. Test strings such as 01, 0011 and 000111.







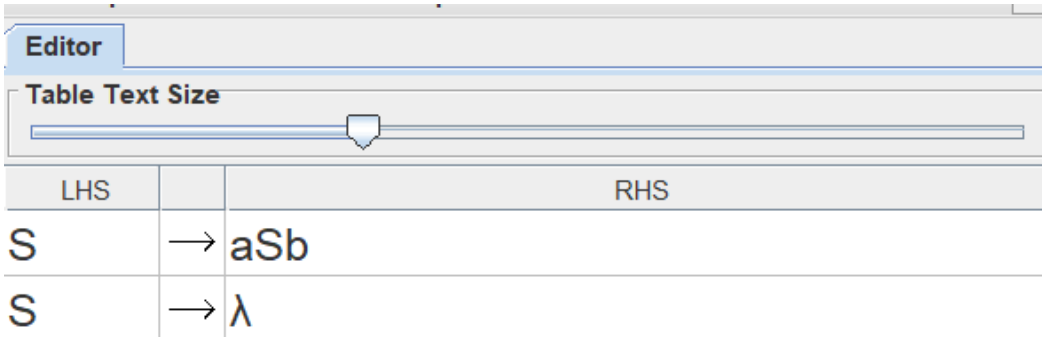
Result: The CFG generates strings made of n zeros followed by n ones.

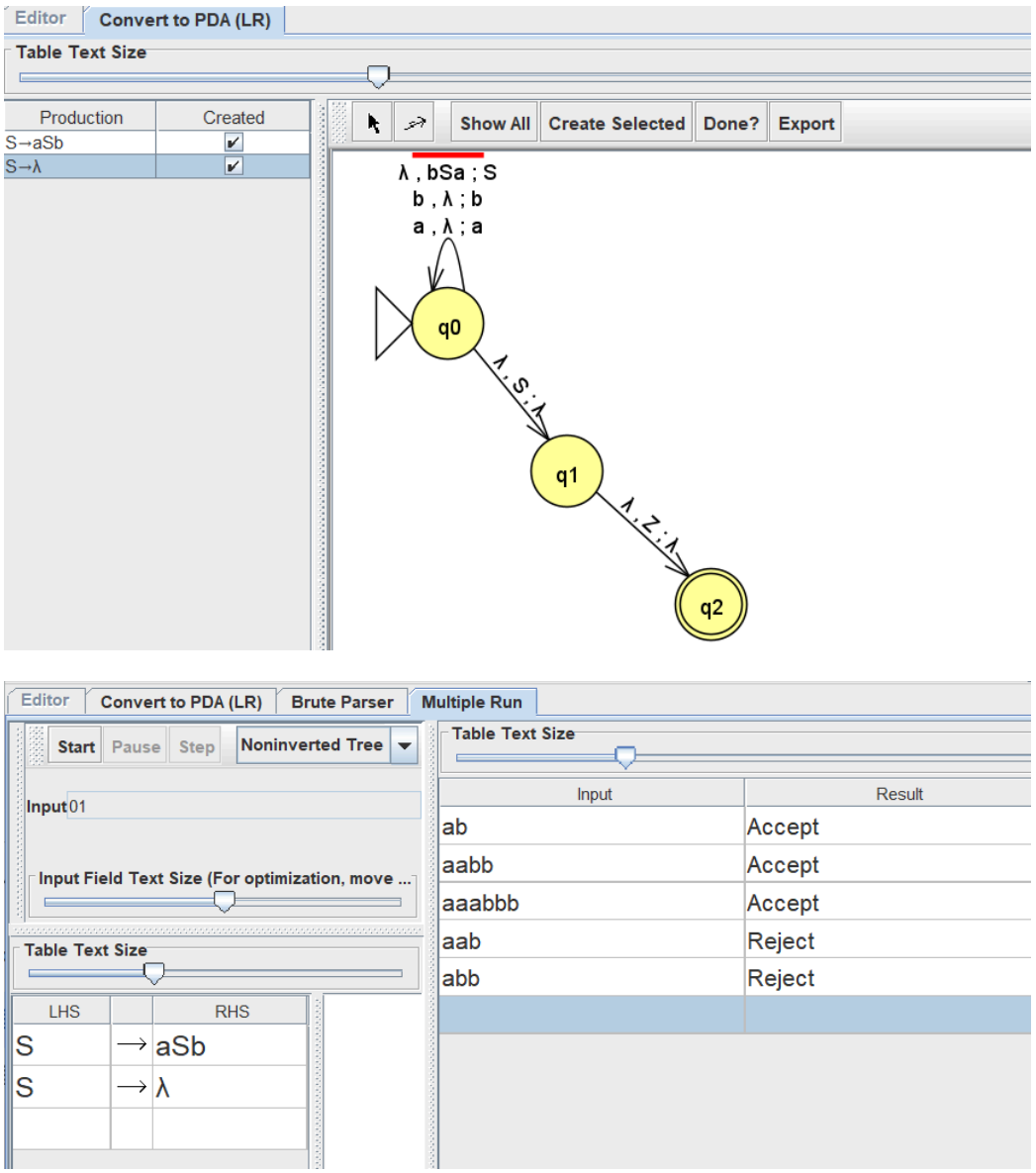
Question 6: LR Parsing

Aim: To explore LR parsing in JFLAP using the grammar and input string from the previous exercise.

Procedure

1. Enter the grammar used in the previous exercise.
2. Open the LR parsing tool.
3. Review the LR items, states, transitions and parsing table that are generated.
4. Parse the input string and record the outcome.





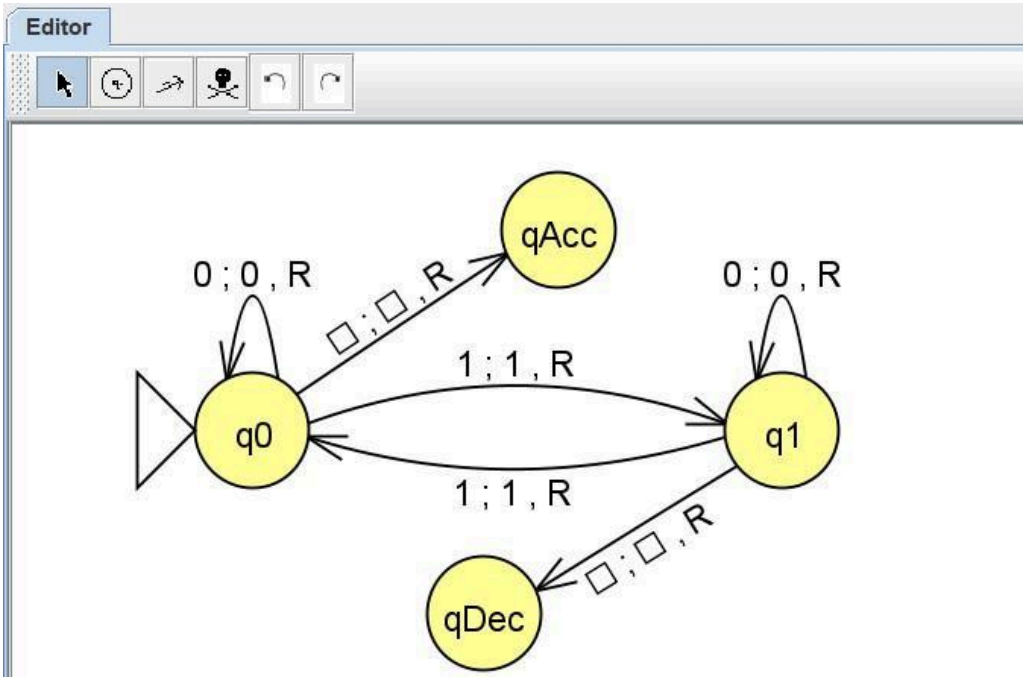
Result: The grammar and input string were analysed using JFLAP's LR parsing feature.

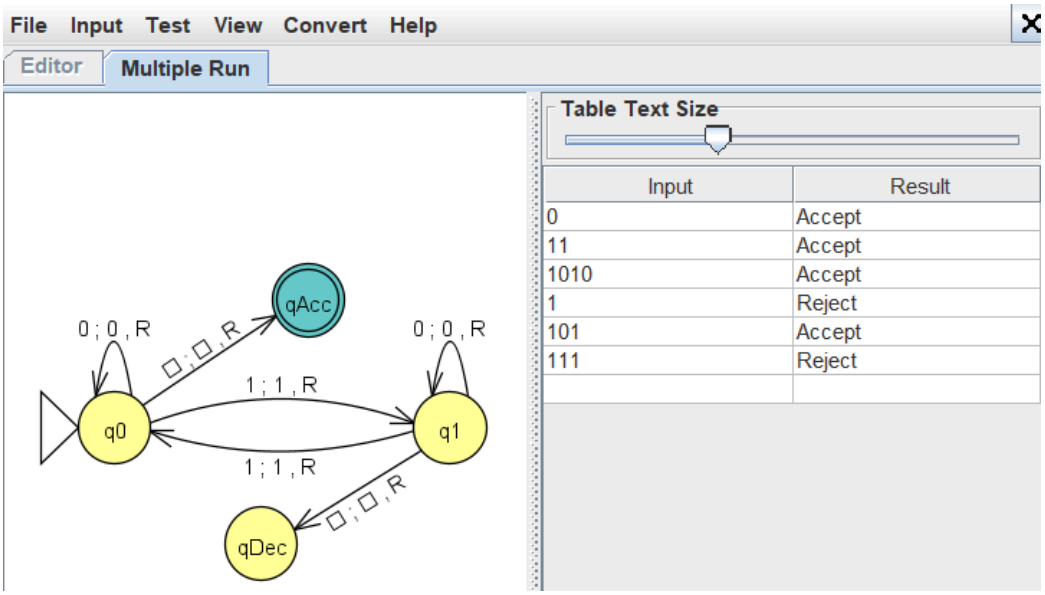
Question 7: One-Tape Turing Machine

Aim: To design a one-tape Turing machine that accepts binary strings containing an even number of 1s.

Procedure

1. Select Turing Machine in JFLAP.
2. Create states that track whether the count of 1s so far is even or odd, together with accepting and rejecting states.
3. Add transitions for the symbols 0, 1 and the blank.
4. Test strings with an even and an odd number of 1s.





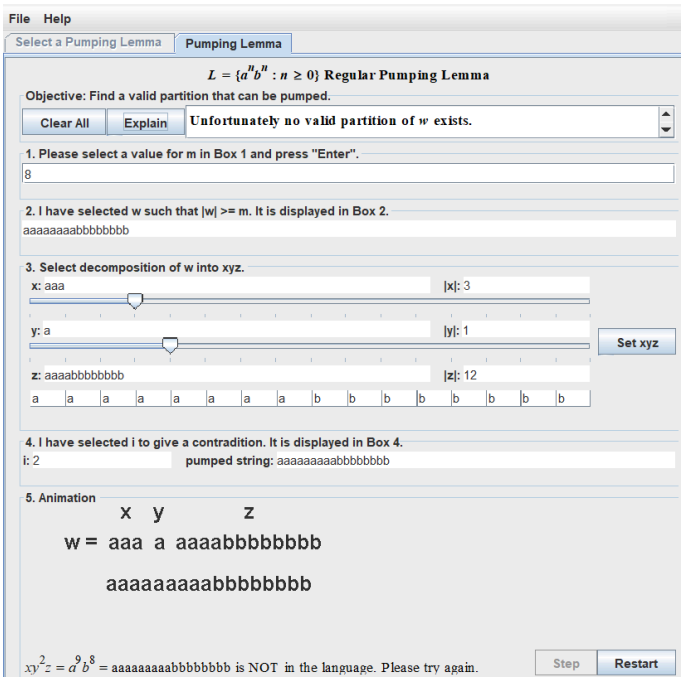
Result: The machine accepts strings with an even number of 1s and rejects those with an odd number of 1s.

Question 8: Regular Pumping Lemma

Aim: To explore the regular pumping lemma in JFLAP using the language $L = \{a^n b^n \mid n \geq 0\}$.

Procedure

1. Open the Regular Pumping Lemma exercise in JFLAP.
2. Select the language (if offered) and enter a pumping length.
3. Follow the prompts to get a string, then choose how to split it into x, y and z.
4. Use the Explain option to see the effect of pumping.



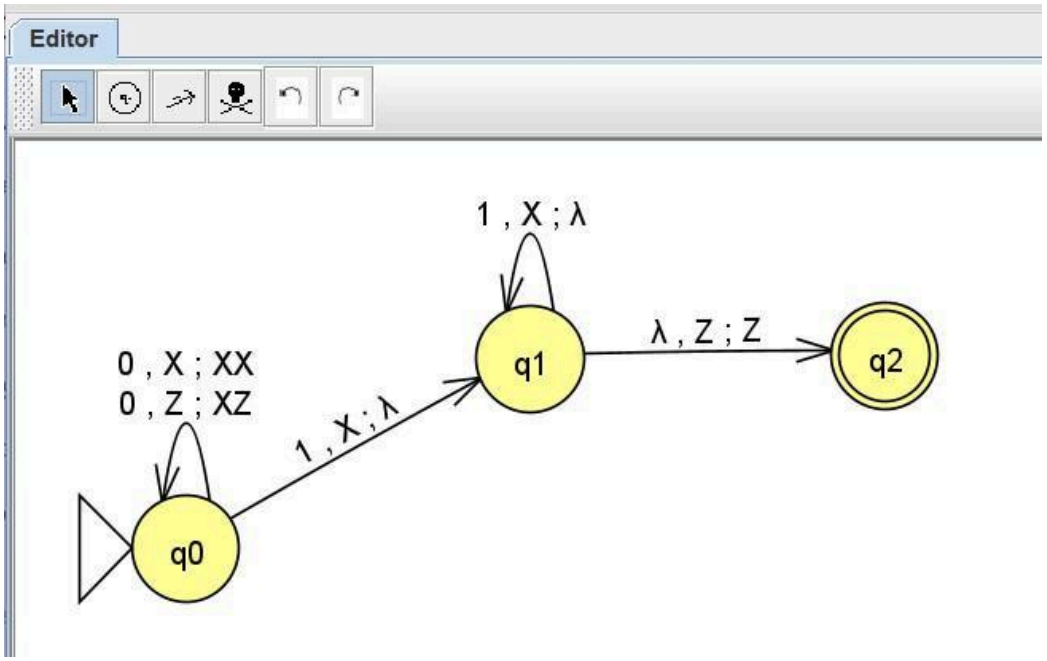
Result: The exercise illustrates the pumping process for a non-regular language: pumping the chosen string produces strings outside L, so the language is not regular.

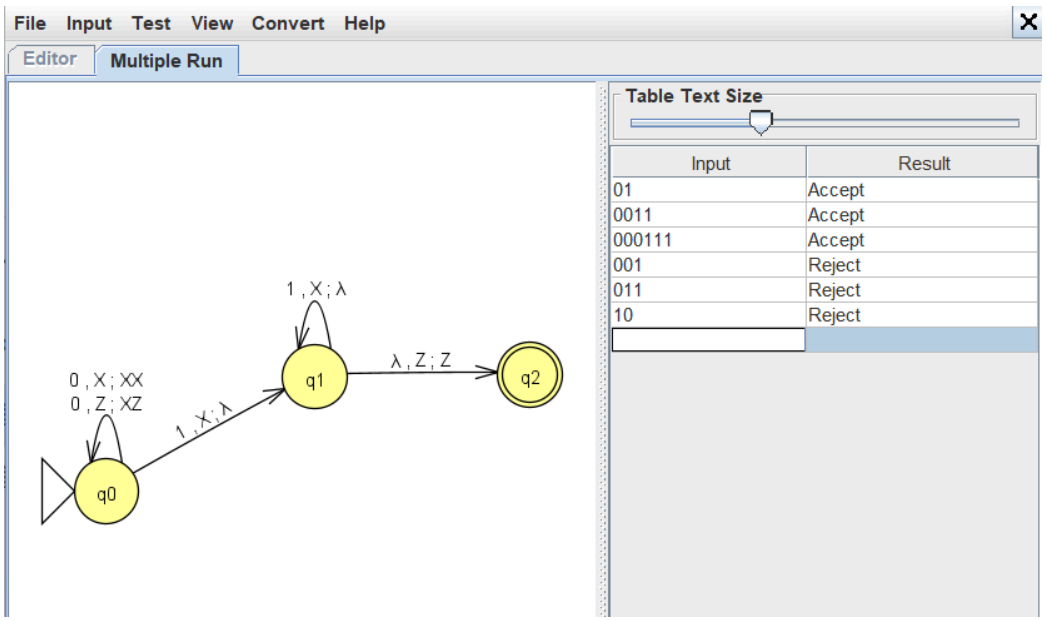
Question 9: NPDA and Conversion to a CFG

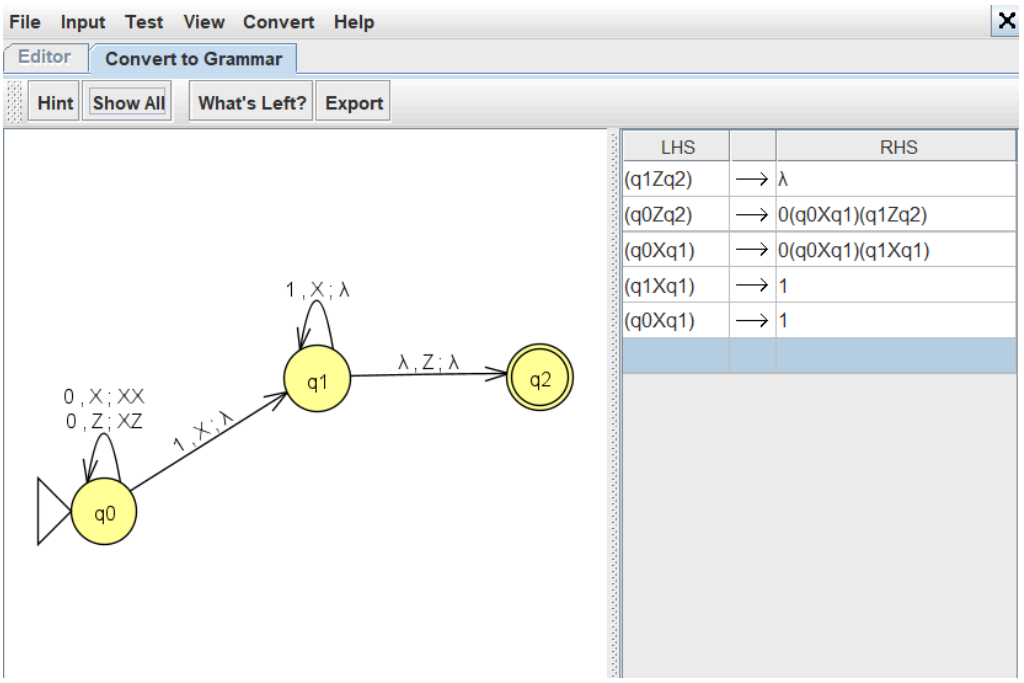
Aim: To construct an NPDA for the language L = $\{0^n1^n \mid n \geq 1\}$ and convert it into an equivalent CFG using JFLAP 7.1.

Procedure

1. Open JFLAP 7.1 and select **Pushdown Automaton**.
2. Create three states, q0, q1 and q2. Set q0 as the initial state and q2 as the final state.
3. Add transitions that push a stack symbol X for every 0 read and pop one X for every 1 read.
4. Add a transition to the final state once all input has been matched and the bottom-of-stack marker is reached.
5. Save the PDA and choose **Convert → Convert to Grammar**.
6. Complete the conversion and examine the grammar produced.



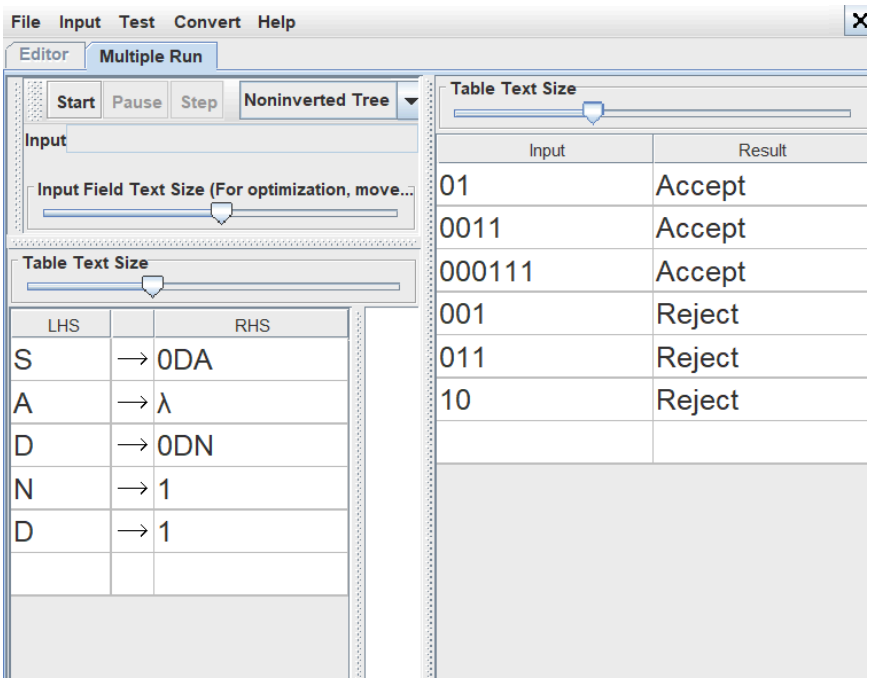

File Input Test View Convert Help
Editor
Multiple Run
Table Text Size
Input
Result
01
Accept
0011
Accept
000111
Accept
001
Reject
011
Reject
10
Reject
1 , X ; λ
λ , Z ; Z
0 , X ; XX
0 , Z ; XZ
1 , X ; λ
q0
q1
q2

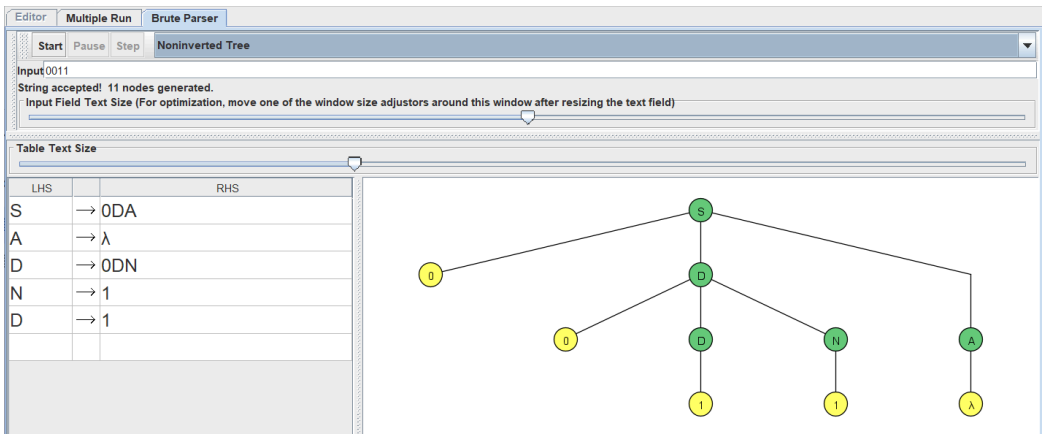

File Input Test View Convert Help
Editor
Convert to Grammar
Hint
Show All
What's Left?
Export
LHS
RHS
(q1Zq2)
λ
(q0Zq2)
0(q0Xq1)(q1Zq2)
(q0Xq1)
0(q0Xq1)(q1Xq1)
(q1Xq1)
1
(q0Xq1)
1
1 , X ; λ
λ , Z ; λ
0 , X ; XX
0 , Z ; XZ
1 , X ; λ
q0
q1
q2

Editor

Table Text Size

LHS		RHS
S	→	0DA
A	→	λ
D	→	0DN
N	→	1
D	→	1





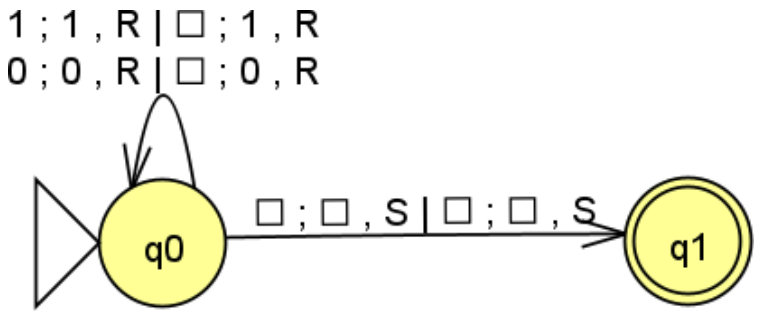
Result: An NPDA was built and tested for L = $\{0^n1^n \mid n \geq 1\}$, and was then converted into a CFG using JFLAP 7.1.

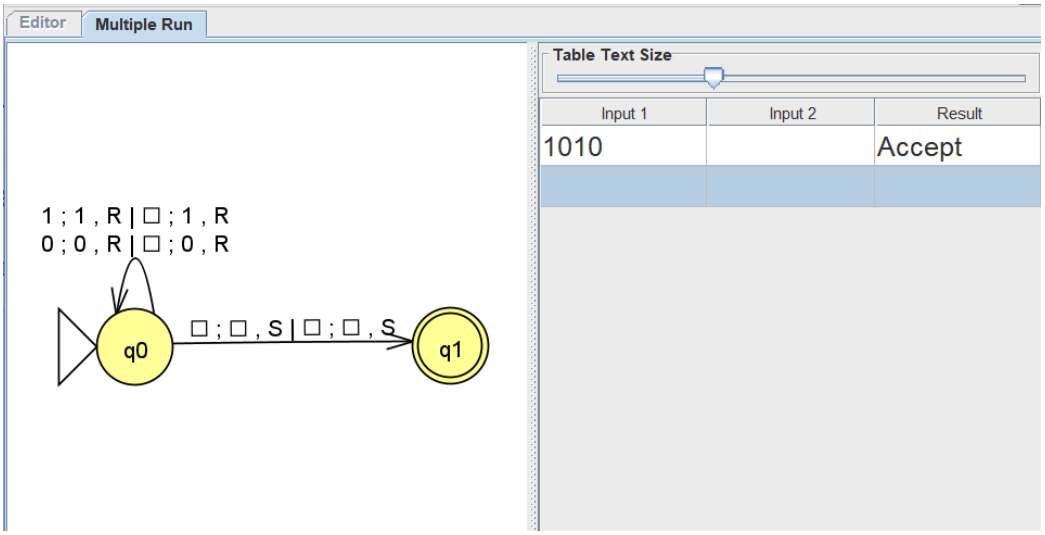
Question 10: Two-Tape Turing Machine

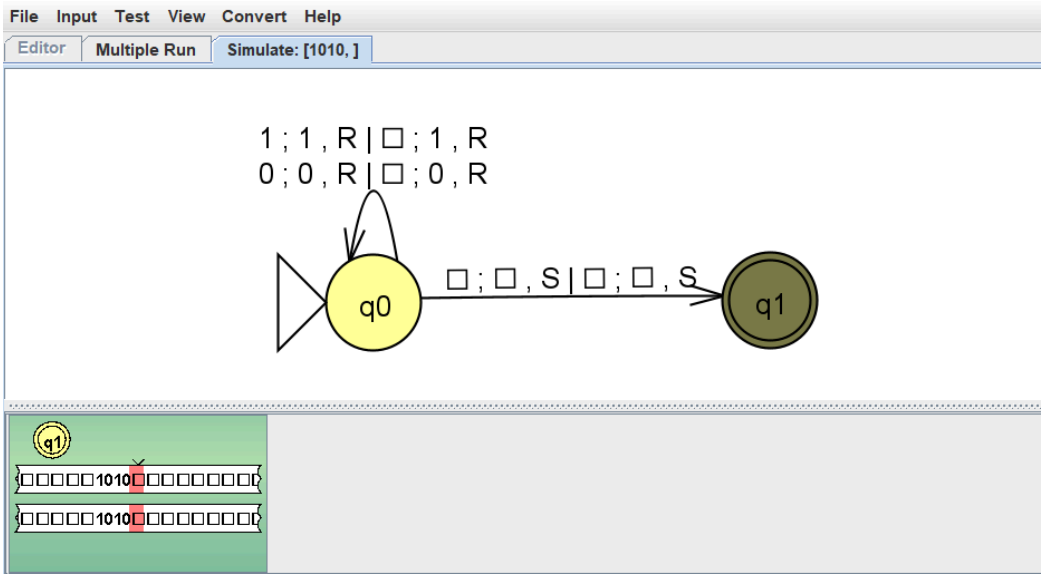
Aim: To design a two-tape Turing machine in JFLAP 7.1 that copies a binary input string from Tape 1 onto Tape 2.

Procedure

1. Open JFLAP 7.1 and select **Multi-Tape Turing Machine**.
2. Create two states, q0 and q1. Set q0 as the initial state and q1 as the final state.
3. Add a self-loop on q0 for input 0: the machine reads 0 on Tape 1, leaves it unchanged, writes 0 on Tape 2, and moves both heads right.
4. Add a second self-loop on q0 for input 1: the machine reads 1 on Tape 1, leaves it unchanged, writes 1 on Tape 2, and moves both heads right.
5. Add a transition from q0 to q1 for when both heads read the blank symbol.
6. Run the machine on the input 1010 and check whether it is accepted.







Result: The two-tape Turing machine was built in JFLAP 7.1 and copied the binary input to Tape 2. It accepted the test input 1010.

Conclusion

JFLAP 7.1 was used to design and simulate DFAs, NFAs, grammars, pushdown automata, parsers and Turing machines, and to convert between equivalent models. The practical showed how these formal models relate to one another and how each can be tested on sample inputs.